

Centro Universitário Filadélfia
Polo Salinópolis
Ciência de Dados e Inteligência Artificial

Aula 5

Programação Orientada a Objetos e Funcional
Fundamentos de Propriedade Intelectual Para a Transferência de
Tecnologia de Software

Thiago Rafael da Silva Moura
trsmoura@yahoo.com.br

28 de novembro de 2025

CAPÍTULO 5

Programação Orientada a Objetos e Funcional

Engenharia de Software para Ciência de Dados
Catherine Nelson

Roteiro da Aula

- > Por que ir além dos Scripts?
- > **Parte 1:** Programação Orientada a Objetos (POO)
- > Classes, Objetos e `'self'`
- > Herança e Composição
- > POO no Scikit-Learn
- > **Parte 2:** Programação Funcional (PF)
- > Funções Puras e Efeitos Colaterais
- > `'map'`, `'filter'` e Comprehensions
- > Comparativo: POO vs PF
- > Abordagem Híbrida em Data Science

Do Notebook para a Produção

Cientistas de dados frequentemente começam com notebooks exploratórios. O código é procedural, linear e cheio de repetições.

Para criar sistemas robustos, escaláveis e testáveis, precisamos adotar paradigmas de engenharia de software: POO e Funcional.

```

100         print('\n-
101         time.sleep(2)
102         start(inventory)
103     else:
104         maintenance(inventory)
105     elif cmd == '3':
106         cargo_hold(inventory)
107     elif cmd == '4':
108         if 'laser' in inventory:
109             print('\n- DECK 4 - DR
110             time.sleep(2)
111             start(inventory)
112         else:
113             droid_hangar(inv
114     if cmd == '5':

```

Dois Paradigmas Principais



Orientada a Objetos

Foca em "**coisas**" (objetos) que contêm dados e comportamentos. Excelente para modelar entidades do mundo real e gerenciar estado.

Funcional

Foca em "**ações**" (funções) e transformação de dados. Excelente para pipelines de dados, processamento paralelo e reprodutibilidade.



Parte 1: Programação Orientada a Objetos

Estruturando código através de Classes e Objetos

Blueprint VS Floor Plan

A Metáfora: Planta vs. Casa

Classe (Class): É a planta (blueprint). Define a estrutura, os atributos e o que pode ser feito. Não existe fisicamente, é apenas um modelo.

Objeto (Object): É a casa construída a partir da planta. Uma instância concreta que ocupa espaço na memória e possui valores específicos.



Anatomia de uma Classe Python

Classes em Python agrupam dados (atributos) e funções (métodos) que operam nesses dados.

```
class ProcessadorDeDados: """Classe para limpar e formatar dados.""" def __init__(self, caminho_arquivo:
```

O Construtor `\_\_init\_\_` e `self`

O método `\_\_init\_\_`

É executado automaticamente quando criamos um novo objeto. É usado para definir o **estado inicial** do objeto.

A variável `self`

Representa a **instância atual** do objeto. É como o objeto diz "minha variável" ou "meu método". Sem `self`, o Python não sabe a qual objeto a variável pertence.

```
def __init__(self, nome): self.nome = nome # Atribui o valor ao objeto específico
```

Atributos vs. Métodos



Atributos

São substantivos. Representam o **estado** ou características.

Ex: `'modelo.acuracia'`, `'cliente.idade'`



Métodos

São verbos. Representam o **comportamento** ou ações.

Ex: `'modelo.fit()'`, `'cliente.comprar()'`



Instanciação

O ato de criar um objeto a partir da classe.

Ex: `'meu_modelo = Modelo(param=10)'`

Exemplo Prático: Engenharia de Features

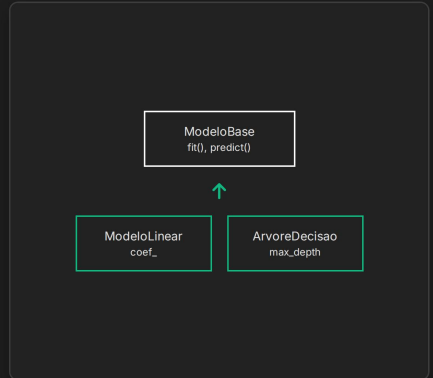
```
import pandas as pd class Normalizador: def __init__(self, metodo="z-score"): self.metodo = metodo self.media = Nc
```

Este padrão permite guardar as estatísticas de treino ('fit') e aplicá-las em teste ('transform').

Herança (Inheritance)

Herança permite criar uma nova classe baseada em uma existente.

- > **Classe Pai (Base):** Define comportamentos genéricos.
- > **Classe Filha (Derivada):** Herda comportamentos e pode adicionar ou modificar funcionalidades.
- > Promove reutilização de código (DRY - Don't Repeat Yourself).



Herança no Código

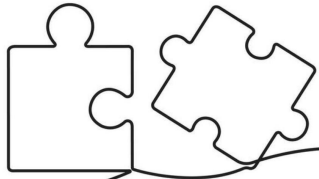
```
class ModeloBase: def salvar(self, caminho): print(f"Salvando modelo em {caminho}") class RedeNeural(Mo
```

Composição vs. Herança

Nem tudo é uma relação "É um" (Herança). Às vezes, a relação é "Tem um" (Composição).

Em vez de herdar complexidade, construímos objetos complexos combinando objetos mais simples. Isso oferece mais flexibilidade.

"Prefira composição à herança." - Design Patterns



Por que isso importa? (Scikit-Learn)

O **Scikit-Learn** é construído inteiramente sobre POO.

- > Todos os algoritmos são **Classes**.
- > Eles herdam de `'BaseEstimator'`.
- > Interface consistente: `'fit()'`, `'transform()'`, `'predict()'`.

Entender POO permite que você crie seus próprios transformadores compatíveis com o Scikit-Learn.

 Tech server room abstract



Parte 2: Programação Funcional

Pipelines de dados, imutabilidade e funções puras

Funções Puras

O coração da programação funcional. Uma função é pura se:

1. > Sempre retorna a mesma saída para a mesma entrada (Determinística).
2. > Não causa **efeitos colaterais** (não altera variáveis globais, não escreve em disco, não altera os argumentos de entrada).

```
# Pura def soma(a, b): return a + b # Impura (depende de estado externo) def soma_impura(a): return a +
```

são?

Alterações de estado que ocorrem "fora" da função. Ex: modificar uma lista passada como argumento.

Por que evitar em Data Science?

Se seus dados mudam silenciosamente enquanto passam por funções, seus notebooks se

⚠ Exemplo Perigoso

```
def remove_outliers(dados): # Modifica a lista original! dados.pop() lista = [1, 2, 100]
```

Funções de Primeira Classe

Em Python, funções são cidadãos de primeira classe. Isso significa que elas podem ser:

Atribuídas

Você pode atribuir uma função a uma variável.

```
meu_func = len
```

Argumentos

Passadas como argumento para outras funções.

```
apply(minha_funcao)
```

Retornadas

Retornadas como resultado de outras funções.

```
return nova_funcao
```

Map e Filter

Substituem loops `for` tradicionais, focando no "o que" fazer, não "como".

Map

Aplica uma função a todos os itens.

```
dados = [1, 2, 3] quadrados = map(lambda x: x**2, dados) # Resultado: [1, 4, 9]
```

Filter

Seleciona itens baseados em uma condição.

```
dados = [1, 2, 3] pares = filter(lambda x: x % 2 == 0, dados) # Resultado: [2]
```

List Comprehensions: O Jeito Pythonico

Muitas vezes preferidas ao `map` e `filter` por serem mais legíveis.

```
# Estilo Imperativo (Loop) resultado = [] for x in dados: if x > 0: resultado.append(x * 2) # Estilo Função
```

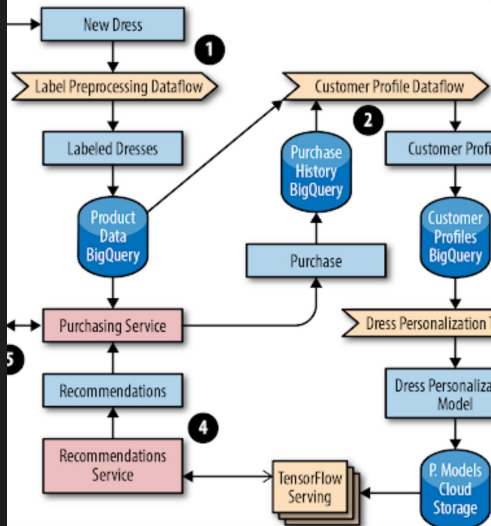
Mais conciso, mais expressivo e frequentemente mais rápido.

Pipelines de Dados

A programação funcional encoraja o encadeamento de operações.

Em vez de mudar o estado linha por linha, os dados fluem através de uma série de transformações, como em uma linha de montagem.

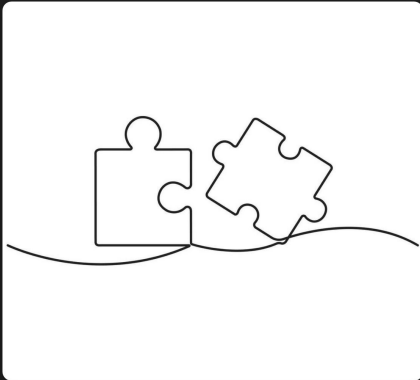
```
dados_limpos =  
dados.pipe(remover_nulos).pipe(normalizar)
```



Comparativo: POO vs. Funcional

Característica	Orientada a Objetos (POO)	Funcional (PF)
Unidade Principal	Objetos (Classe)	Funções
Estado	Mantido dentro dos objetos (mutável)	Evitado ou passado via args (imutável)
Dados vs. Comportamento	Agrupados juntos	Separados
Uso em Data Science	Criar modelos, APIs, estimadores	ETL, limpeza de dados, pipelines

Abordagem Híbrida em Data Science



O Melhor dos Dois Mundos

Não precisamos escolher apenas um. Python é multiparadigma.

- > Use **Funcional** para o pré-processamento de dados e transformações estatísticas (stateless).
- > Use **POO** para gerenciar o modelo de Machine Learning, configurações e conexões de banco de dados (stateful).

Resumo: Escrevendo Código Melhor



Modularidade

Classes e funções pequenas tornam o código mais fácil de entender e manter.



Testabilidade

Funções puras e classes bem definidas são muito mais fáceis de testar.



Reutilização

Herança e funções genéricas evitam duplicação de código.



Dúvidas?

Obrigado!